df ['num_words'] = df ['text'].apply (lambda x: len (nltk.word_tokenize (x)))

New column called num_words added to Dataframe df, where no. of words in each entry from text column is calculated.

(1) df ['text']: This accesses the column text in the Dataframe df. This column is assumed to contain text data (strings).

(2) apply (lambda x: ...): The apply function is used to apply a given funⁿ ~~along the axis (rows in this case) of the Dataframe.~~
~~The funⁿ being applied here is a lambda funⁿ, which takes x as an argument.~~
~~The x represents each individual words (tokens) for ex, if x was "Hello, world!", it would return a list like ['Hello', ',', 'world', '!'].~~

(3) nltk.word_tokenize (x):
The nltk.word_tokenize() function is a part of NLP Toolkit (NLTK), a library in python used for NLP.

(2) ... lambda function, which takes as an argument. The x represents each individual entry (a string of text) in the text column.

(4) len (nltk.word_tokenize (x)): This computes the length of the list of tokens (words) are in the string x.
Note: punctuation marks like commas & periods are counted as seperate token.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
T	a	b	a	b	c	a	b	c	a	b	a	b	a	b	d

pattern : a b a b d

⇒ (1) Prepare π table for pattern

1	2	3	4	5
a	b	a	b	d
0	0	1	2	0

(2) Take 2 variables i at the 1st index of text & j at π table before 1st index.

We will compare i with j+1 position.

i

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	b	a	b	c	a	b	c	a	b	a	b	a	b	d

j+1

j

	1	2	3	4	5
0	a	b	a	b	d
	0	0	1	2	0

if i = j+1 (if match then move/increment i & j)
then i++, j++ (i=1, j=0)

i=j=0 compare i = j+1
i++ , j++

i=j=1 i+1 = j+2
i++ , j++

i=j=2 i+2 = j+3
i++, j++

i=j=3 i+3 = j+4
i++, j++

i=j=4 i+4 ≠ j+5
j --
j is indexing at (2)
j = 2

j

1	2	3	4	5
a	b	a	b	d
0	0	1	2	0

i

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	b	a	b	c	a	b	c	a	b	a	b	a	b	d

Not matched

J backtrack

0	1	2	3	4	5
	a	b	a	b	d
	0	0	1	2	0

i

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	b	a	b	c	a	b	c	a	b	a	b	a	b	d

0	1	2	3	4	5
	a	b	a	b	d
	0	0	1	2	0

Now increment it because cant backtrack

i i i

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	b	a	b	c	a	b	c	a	b	a	b	a	b	d

Not matching

j 0	j+1 1	2 j	3 i	4	5
	a	b	a	b	d
	0	0	1	2	0

→ Now $\boxed{\text{increment } i}$

i

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	b	a	b	c	a	b	c	a	b	a	b	a	b	d

j 0	j+1 1	2 j	3	4	5
	a	b	a	b	d
	0	0	1	2	0

Rabin Karp

Rabin-karp algorithm is an algorithm used for searching / matching patterns in the text using a hash function.
Unlike Naive string matching algo., it does not travel through every character in the initial phase rather it filters the characters that do not match and then performs the comparision.

Hash function → A hash function is a tool to map a larger input value to a smaller output value, is called the hash value.

Algorithm :-

```
n = t.length
m = p.length
h = dm-1 mod q
p = 0
t0 = 0
for i = 1 to m
    p = (dp + p[i]) mod q
    t0 = (dt0 + t[i]) mod q
For s = 0 to n-m
    if p = ts
        if p[1 ... m] = t[s+1 ... s+m]
            print "pattern found at position" s
    if s < n-m
        ts+1 = (d (ts - t[s+1]h) +
                t[s+m+1]) mod q
```

The hashcodes of a substring of text are matching with the hashcode of the pattern but both strings are not the same. In this way we are comparing patterns every time and it is not matching every time
· If we take such a simple Hash function, there's a possibility of collision with the other substrings which are having the same

code through they are not patterns.
This is also called spurious Hits.
So the maximum time taken by algo.
Worst case $O(m \times n)$
We selected such a type of hash function which is causing such spurious Hits.
So we need to change the Hash funⁿ.

Hash function =
$(\text{pattern}[0] * (x^{m-1})) + (\text{patter}[1] * (x^{m-2})) + (\text{pattern}[2] * (x^{m-3})) + \dots \text{pattern}[m-1) * (x^0))$
% mod

Here, x = base value (No. of alphabets used if 26 alphabets are used in the text, then $x = 26$)
m = length of pattern

Time Complexity
* Average & ~~worst~~ Best
  $O(n+m)$
* Worst case
  $O(n * m)$
* Space Complexity $O(1)$

text : abcdefghijk

pattern : cdef

sliding window

a	1	j	10
b	2	k	11
c	3	l	
d	4		
e	5		
f	6		
g	7		
h	8		
i	9		

$h(cdef) = h(3,4,5,6) = 18$

$h(abcd) = h(1,2,3,4) = 10$

$h(bcde) = h(abcd) - 1 + 5$

$= 10 - 1 + 5 = 14$

$h(cdef) = h(bcde) - 2 + 6$

$= 14 - 2 + 6 = 18$

multiplier $= 10^{n-1}$

where $n \rightarrow$ character position from right

$h(abcd) = h(1,2,3,4)$

$= (\underbrace{1 * 10^3}_{a} + 2 * 10^2 + 3 * 10^1 + 4 * 10^0) \bmod 113$

$= (\underbrace{1000}_{a} + \underbrace{200}_{b} + \underbrace{30}_{c} + \underbrace{4}_{d}) \bmod 113$

$= (1234) \bmod 113$

$h(bcde) = ((h(abcd) - \underbrace{1 * 10^3}_{a}) * 10 + 5) \bmod 113$

$= (1234 - 1000) * 10 + 5 = (2324) \bmod 113$

$h(cdef) = (h(bcde) - \underbrace{2 * 10^3}_{b}) * 10 + 6$

$= (2345 - 2000) * 10 + 6 = 3456$

$h(abcd) = (1234) \bmod 113 = 104$

pick any prime number.

$h(bcde) = ((((h(abcd) - 1 * 10^3 \bmod 113) * 10) \bmod 113 + 5) \bmod 113$

$(((104 - 96) * 10) \bmod 113 + 5) \bmod 113$

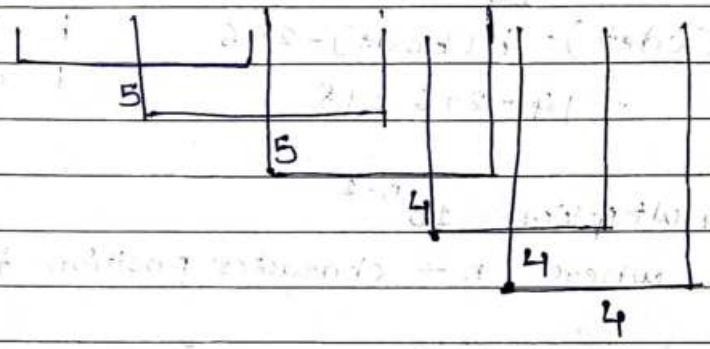
$(80 \bmod 113 + 5) \bmod 113 = 85$

T =	a	c	a	a	b	a	a

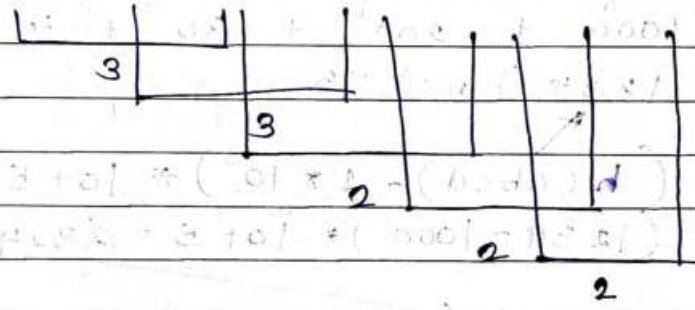
P =	a	a	b
	1	1	2

a = 1 c = 3
b = 2 d = 4

	a	c	a	a	b	a	a
T	1	3	1	1	2	1	1



1 3 1 1 2 1 1



Choose any random prime no. q

p [.....] mod q

Knuth- Morris - Pratt Algorithm

The KMP algorithm is used to solve the pattern matching problem which is a task of finding all the occurrences of a given pattern in a text. It is useful when it comes to finding multiple patterns.

For ex → if the text is
text → aabbaaccaabbaadde
and the pattern is → aabaa
Then the pattern occurs twice in the text, at indices 0 and 8.

The brute force & naive approaches may result in redundant comparisions.
To avoid such redundancy, Knuth-Morris-Pratt developed a linear sequence-matching algorithm named the KMP pattern matching algorithm.

Working of the algorithm-
The KMP algorithm starts the search operation from left to right. It uses the prefix function to avoid unnecessary comparisions while searching for the pattern.
This function stores the no. of characters matched so far which is known as LPS value.
↖ Longest proper prefix | suffix

Steps of KMP algo:-
1. Define a prefix function
2. Slide the pattern over the text for comparision.
3. If all the characters match, we have found a match.
4. If not, use the prefix function to skip unnecessary comparisions.
   ↳ If the LPS value of previous character from the mismatched character is '0', then start comparision from index 0 of pattern with the next character in the text.
   However, if the LPS value is more than '0', start the comparision from index value equal to LPS value of the previously mismatched character.

π table longest prefix that same as suffix (LPS)

index →	1	2	3	4	5	6	7	8	9	10
1.	a	b	c	d	a	b	e	a	b	f
π Table →	0	0	0	0	1	2	0	1	2	0

Repeated
so write index
of a

	1	2	3	4	5	6	7	8	9	10	11
2.	a	b	c	d	e	a	b	f	a	b	c
π table	0	0	0	0	0	1	2	0	1	2	3

	1	2	3	4	5	6	7	8	9
3.	a	a	a	a	b	a	a	c	d
π table	0	1	2	3	0	1	2	0	0

	1	2	3	4	5	6	7	8	9	10
4.	a	b	a	b	a	b	a	b	c	a
π table	0	0	1	2	3	4	5	6	0	1